

Project: Secure File Sharing with S3 and CloudFront

Services:

Amazon S3, CloudFront, IAM, API, Lambda@Edge

Description:

Implement a secure file-sharing service where users can upload files to Amazon S3, and CloudFront securely serves files using signed URLs. Lambda@Edge is used to validate requests and improve security for content delivery.

Skills:

CDN setup, secure file access, IAM policies, CloudFront signed URLs, Lambda@Edge configuration.

Definition:

The Secure File Sharing with S3 and CloudFront project is designed to provide secure and scalable file distribution using AWS cloud services. Files are stored privately in Amazon S3 and delivered securely through Amazon CloudFront using signed URLs. Access to files is restricted so that only authorized users with valid signed URLs can download or view the files.

Scope of the Project:

The scope of this project is to design and implement a secure file-sharing architecture using AWS services. The system allows secure storage and controlled delivery of files while preventing unauthorized access.

- Secure File Storage using Amazon S3.
- Controlled File Delivery using CloudFront CDN.
- API Gateway is mainly used to provide a secure backend endpoint for generating signed URLs and handling file uploads/downloads.
- Signed URL generation for temporary secure access.
- Lambda@Edge integration for request validation.
- IAM-based secure access policies.
- Private S3 bucket configuration.
- Prevention of direct S3 public access.

Methodology:

1. S3 + CloudFront + Lambda@Edge – Best Method

- Uses all required AWS services: S3, CloudFront, IAM, API and Lambda@Edge.
- Provides secure and scalable file delivery architecture.
- Uses signed URLs to restrict unauthorized access.
- Lambda@Edge validates requests at CloudFront edge locations.
- S3 bucket remains private and inaccessible directly from the internet.

2. S3 Public Access + Application Authentication – Alternative Method

- Files are stored in S3 with public access enabled.
- Authentication handled only at application level.
- Easier to implement but less secure.
- Suitable only for low-security workloads.

AWS Services Used in the Project:

1. Amazon S3 (Simple Storage Service)

Purpose: Stores files securely in private buckets.

2. Amazon CloudFront

Purpose: Distributes files globally with secure access control.

3. AWS IAM (Identity and Access Management)

Purpose: Manages permissions for CloudFront and Lambda services.

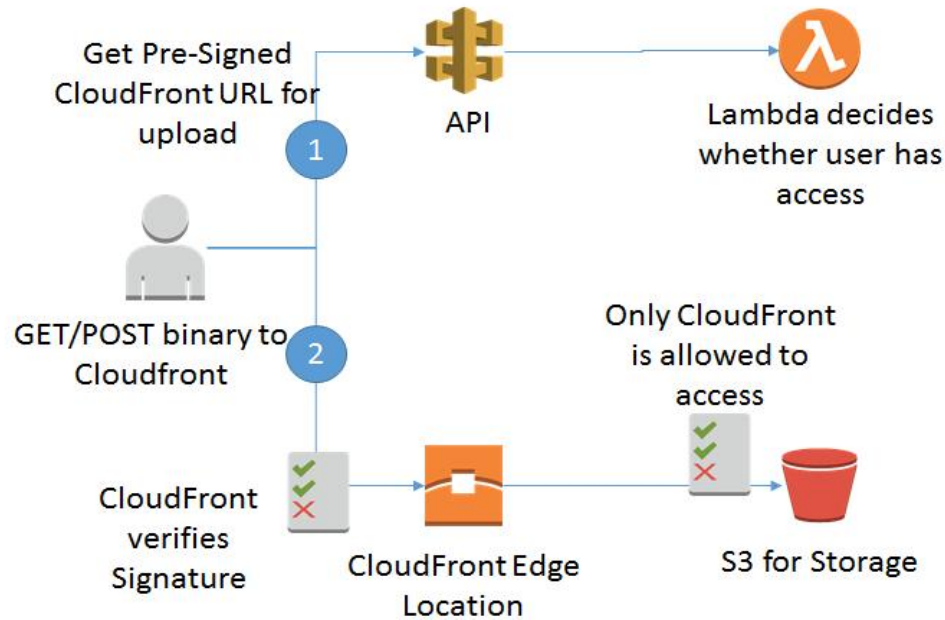
5. AWS API Gateway

Purpose: API Gateway is mainly used to provide a secure backend endpoint for generating signed URLs and handling file uploads/downloads.

4. AWS Lambda@Edge

Purpose: Adds custom authentication and request validation.

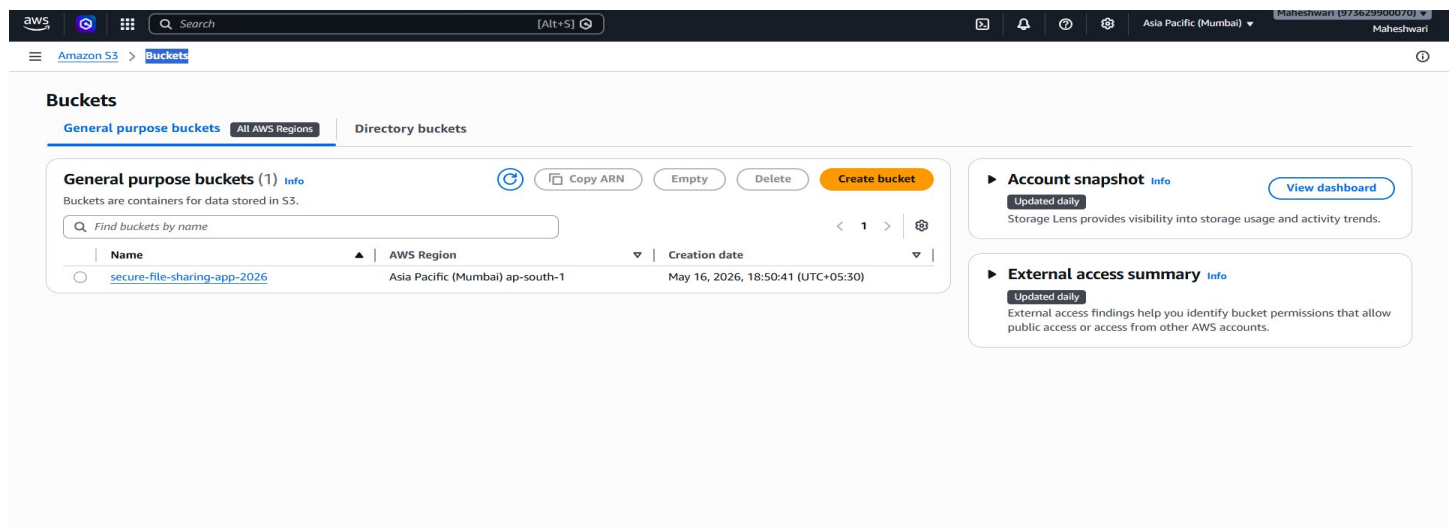
Architecture Diagram:



Implementation Steps:

Step 1: Create S3 Bucket

- Go to AWS Console → S3 → Create Bucket.
- Bucket Name: secure-file-sharing-app-2026
- Disable public access settings.
- Upload swap1.pdf



secure-file-sharing-app-2026 Info

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Name	Type	Last modified	Size	Storage class
swap1.pdf	pdf	May 18, 2026, 01:31:12 (UTC+05:30)	1.9 MB	Standard

Step 2: Create CloudFront Distribution

- Go to CloudFront → Create Distribution.
- Select S3 bucket as origin.
- Configure Origin Access Control (OAC).
- Redirect HTTP to HTTPS.

cloudfront-s3 Standard [View metrics](#)

Details

Distribution domain name d3divbu6dbw7fq.cloudfront.net	Billing Pay-as-you-go Switch to a plan	ARN arn:aws:cloudfront::973629900070:distribution/E193M65AF0FD9S	Last modified May 17, 2026 at 7:09:22 PM UTC
--	---	--	--

General Security Origins Behaviors Error pages Invalidations Logging Tags

Settings [Edit](#)

Name cloudfront-s3 ✎	Alternate domain names - Add domain	Standard logging Off
Description -		Cookie logging Off

Step 3: Update S3 Bucket Policy

- Copy CloudFront-generated bucket policy.
- Paste policy in S3 Bucket Permissions.

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

Amazon S3 > Buckets > secure-file-sharing-app-2026

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

[Edit](#) [Delete](#)

```
{
  "Version": "2008-10-17",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::secure-file-sharing-app-2026/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::973629900070:distribution/E193M65AF0FD9S"
        }
      }
    }
  ]
}
```

[Copy](#)

Object Ownership

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

[Edit](#)

Step 4: Create IAM Role for Lambda@Edge

- Attach AWSLambdaBasicExecutionRole.
- Attach CloudFrontFullAccess.

aws [Search] [Alt+S] Global Maheshwari (973629900070) Maheshwari

IAM > Roles > uploadFileAndGenerateUrl-role-xhebwqzu

uploadFileAndGenerateUrl-role-xhebwqzu

[Delete](#) [Edit](#)

Summary

Creation date
May 17, 2026, 12:59 (UTC+05:30)

Last activity
[23 hours ago](#)

ARN
[arn:aws:iam::973629900070:role/service-role/uploadFileAndGenerateUrl-role-xhebwqzu](#)

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (3)

You can attach up to 10 managed policies.

[Simulate](#) [Remove](#) [Add permissions](#)

Filter by Type
All types

Identity and Access Management (IAM)

[Search IAM](#)

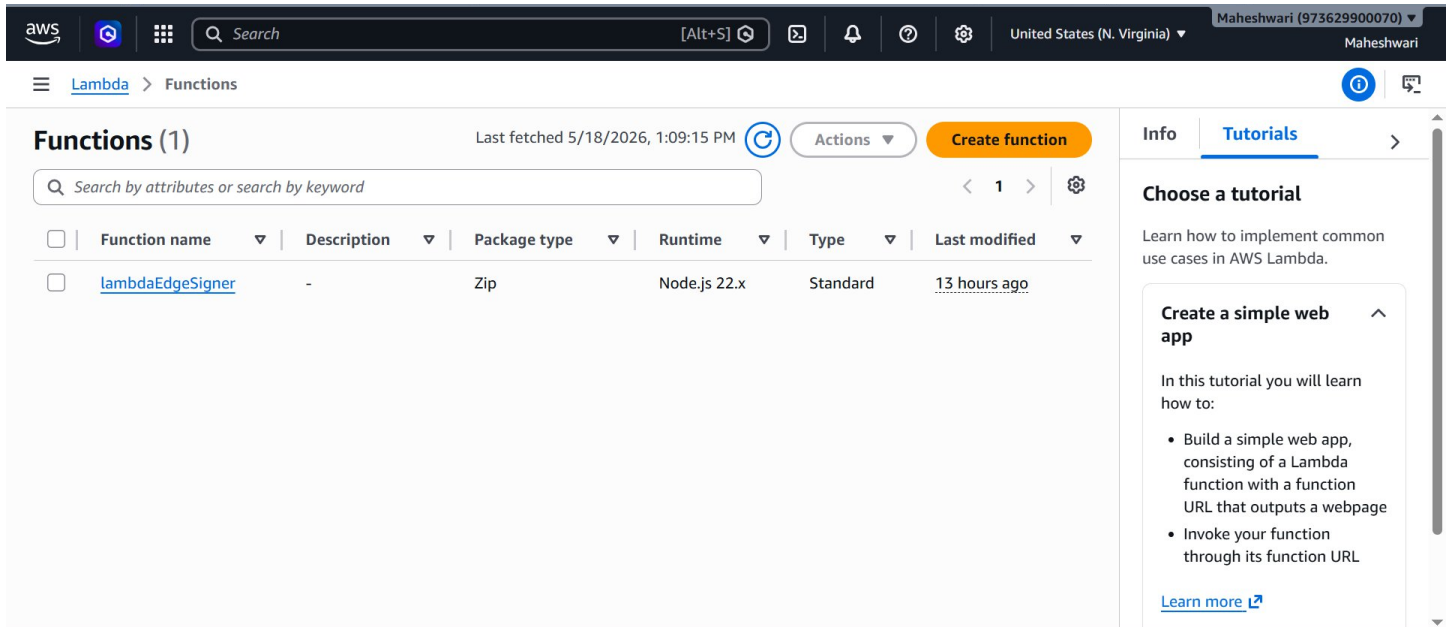
Access Management

- [Roles](#)
- [Policies](#)
- [IAM users](#)
- [IAM user groups](#)
- [Identity providers](#)
- [Account settings](#)
- [Root access management](#)
- [Temporary delegation requests](#)

Access reports

Step 5: Create Lambda@Edge Function

- Region: us-east-1
- Runtime: Python 3.10



The screenshot shows the AWS Lambda console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and various utility icons. Below the navigation bar, the breadcrumb trail shows 'Lambda' > 'Functions'. The main content area is titled 'Functions (1)' and shows a table with one function listed: 'lambdaEdgeSigner'. The table columns are: Function name, Description, Package type, Runtime, Type, and Last modified. The function 'lambdaEdgeSigner' has a description of '-', package type of 'Zip', runtime of 'Node.js 22.x', type of 'Standard', and was last modified '13 hours ago'. To the right of the table, there's a sidebar with 'Info' and 'Tutorials' tabs. The 'Tutorials' tab is active, showing a section titled 'Choose a tutorial' with a list of tutorials. The first tutorial is 'Create a simple web app', which includes a description and a list of steps: 'Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage' and 'Invoke your function through its function URL'. A 'Learn more' link is also present.

Function name	Description	Package type	Runtime	Type	Last modified
lambdaEdgeSigner	-	Zip	Node.js 22.x	Standard	13 hours ago

Step 6: Add Lambda Function Code

- Deploy Lambda function.

Step 7: Deploy Lambda@Edge

- Associate Lambda function with Viewer Request.

Step 8: Configure CloudFront Behavior

- Attach Lambda@Edge under Function Associations.

Step 9: Create Upload API

Create Amazon API Gateway

Add Lambda Integration

Deploy API

API Gateway > APIs > apicloudfront (wxb0dqcyu2) > Stages

API Gateway

- APIs
- Custom domain names
- Domain name access associations
- VPC links

API: apicloudfront... (wxb0dqcyu2)

▼ Develop

- Routes
- Authorization
- Integrations
- CORS
- Reimport
- Export

▼ Deploy

- Stages

▼ Monitor

- Metrics

Stages

Stages for apicloudfront [Create](#)

Search

- prod

Stage details

[Delete](#) [Edit](#)

Details

Name	prod	Created	May 17, 2026 7:11 PM	Last updated	May 17, 2026 7:11 PM
-------------	------	----------------	----------------------	---------------------	----------------------

Invoke URL

<https://wxb0dqcyu2.execute-api.ap-south-1.amazonaws.com/prod>

Description

None

Attached deployment

Automatic Deployment

Enabled

Deployment ID	pcdiaq	Deployment created	May 17, 2026 7:11 PM
----------------------	--------	---------------------------	----------------------

Deployment description

Automatic deployment triggered by changes to the Api configuration

Stage variables

Find resources

< 1 >

Step 10: Generate Signed URL

- Generate CloudFront signed URLs.

ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions

Functions (1)

Last fetched 5/18/2026, 1:13:24 PM [Actions](#) [Create function](#)

Search by attributes or search by keyword

Function name	Description	Package type	Runtime	Type	Last modified
uploadFileAndGenerateUrl	-	Zip	Node.js 22.x	Standard	12 hours ago

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari

Lambda > Functions > uploadFileAndGenerateUrl

Layers (0)

API Gateway

+ Add trigger

+ Add destination

Last modified 12 hours ago

Code Test Monitor Configuration Aliases Versions

Executing function: succeeded (logs)

Details

```
{
  "statusCode": 200,
  "body": "{\"signedUrl\":\"https://d3d1vbu6dbw7fq.cloudfront.net/swap1.pdf?Expires=1779093718&Key-Pair-Id=KJOP7CKAEHNG&Signature=pk1v1RK12uVMEAbd67vh3lpudM8j3W6gH-1PHj1KBm6CVj0TqgEJ1Q8B18-2p5oyPzNzmmu9zQt-y3JuU7ryyHag7yWoaAgpVQ3fNlUcO3nlnfxvap7eVyD8kLzBqQzARyYHwS5loqsR311fVzh2jN2tmqZqHKnFUFVNBP15Keoc6rjtUHQvVX6YfuzfKlabXBKSUY12HwA-paXvrtqY1qPDWc-fly9KH167b9HncVgJgVnbA1C-AdC1n2712dLwLcY1DCLC1otkkFH-T-NIQCifwa2YguV21ahQyY36mMSJv8PobiH0IDOp7rf6-y879u49T1HJbyUG-2dGzu_\"}"
}
```

Summary

Code SHA-256	Execution time
QaENUC2EEKOTXgDhb8zrGASIM6aG2g/Ov/Rk8sr588=	11 seconds ago
Function version	Request ID
\$LATEST	7164e4f5-f555-450a-a339-65d2919a90e7
Duration	Billed duration
11.2 s	11.2 s

Step 10: Test the Project

- Verify direct access is denied.

secure-file-sharing-app-2026.s3.ap-south-1.amazonaws.com/swap1.pdf

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0"?>
<Error>
  <Code>AccessDenied</Code>
  <Message>Access Denied</Message>
  <RequestId>0M0QVPMHX7F3V4PG</RequestId>
  <HostId>ofBmpC32ch5irdwCvNvMEKmw6hnc1bLtsjLLkTtHmCnqn8Lxon1OnciXDH1/AYkqdtzp1EpnQ=</HostId>
</Error>
```

- Verify signed URL grants access.

d3divbu6dbw7fq.cloudfront.net/swap1.pdf?Expires=1779050487&Key-Pair-Id=KJOP7CKAEHNG&Signature=T84~EELafDL...

swap1.pdf

1 / 3 85%

1

2

3

Launch an instance | Launch an instance | Instances | EC2 | ap-south-1 | EC2 Instance Connect | Volumes | EC2 | ap-south-1 | Ask Gemini

ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0dc06773aea7...

```
aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari Maheshwari

/m/
Last login: Thu Apr 30 04:07:55 2026 from 13.233.177.4
[ec2-user@ip-172-31-42-249 ~]$ sudo su
[root@ip-172-31-42-249 ec2-user]# fdisk /dev/nvme1n1

Welcome to fdisk (util-linux 2.37.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0xcb77fe67.

Command (m for help): n
Partition type
  p   primary (0 primary, 0 extended, 4 free)
  e   extended (container for logical partitions)
Select (default p):

Using default response p.
Partition number (1-4, default 1):
First sector (2048-8388607, default 2048):

i-0dc06773aea7d282d (myserver)
PublicIPs: 43.205.231.220 PrivateIPs: 172.31.42.249
```


Troubleshooting:

Issue 1: AccessDenied Error

Cause: Incorrect bucket policy or OAC configuration.

Solution: Verify CloudFront OAC and bucket policy.

Issue 2: Missing Key-Pair-Id Error

Cause: Missing signed URL parameters.

Solution: Ensure URL contains Expires, Signature, and Key-Pair-Id.

Issue 3: Lambda Function Not Triggering

Cause: Lambda@Edge not associated properly.

Solution: Associate Lambda@Edge under Viewer Request.

Conclusion:

The Secure File Sharing with S3 and CloudFront project provides a scalable and secure solution for controlled file distribution using AWS cloud services. Files stored in private S3 buckets are securely delivered through CloudFront using signed URLs, ensuring that only authorized users can access the content. The integration of Lambda@Edge enhances security by validating requests at edge locations, while IAM policies and OAC prevent direct access to S3 objects.